PROJECT REPORT

PLANTS VS ZOMBIES

Java Swing Game Development

Group: AB

Member: (Student - ID)

Thái Nhật Anh – ITCSIU25004

Nguyễn Tri Nhân - ITCSIU25040

Nguyễn Lê Hoàng Long - ITCSIU25030

Dương Tấn Lực - ITCSIU25033

Abstract

This project implements a simplified version of the popular tower defense game **Plants vs. Zombies** using Java and Swing. The game features multiple plant types, different zombie variants, projectile mechanics, resource (sun) management, and defensive systems such as Lawn Mowers. The objective is to defend the lawn from waves of zombies using strategic plant placement. The project demonstrates objectoriented programming principles, GUI development, animation handling, and collision detection.

1. Introduction

Plants vs. Zombies is a classic tower defense game where players must prevent zombies from reaching their house by planting defensive plants. This project recreates the core gameplay mechanics in Java using the Swing library for the graphical user interface.

Objectives:

- Implement core game mechanics (planting, shooting, zombie movement, collision).

- Apply Object-Oriented Programming (OOP) concepts effectively.

- Create an engaging and responsive graphical game.

- Demonstrate proper software design through class hierarchy and inheritance.

2. Game Rules

- The game is played on a 5-row lawn with an 8-column grid.
- Sun is the main resource, generated by Sunflower/Sunshroom and falling randomly from the sky.
- Players collect sun and select plants from the seed packet bar to place on the grid.
- Zombies move from right to left. If any zombie reaches the leftmost side, the player loses.
- Different plants have unique abilities (shooting, exploding, freezing, hypnotizing, eating, etc.).
- Lawn Mowers automatically activate to clear an entire lane when zombies get too close.
- The game ends when all lawn mowers are used and a zombie reaches the house.

3. System Design

3.1 Class Diagram

The system follows a clean object-oriented design with proper use of inheritance and composition.

Main Class Hierarchy:

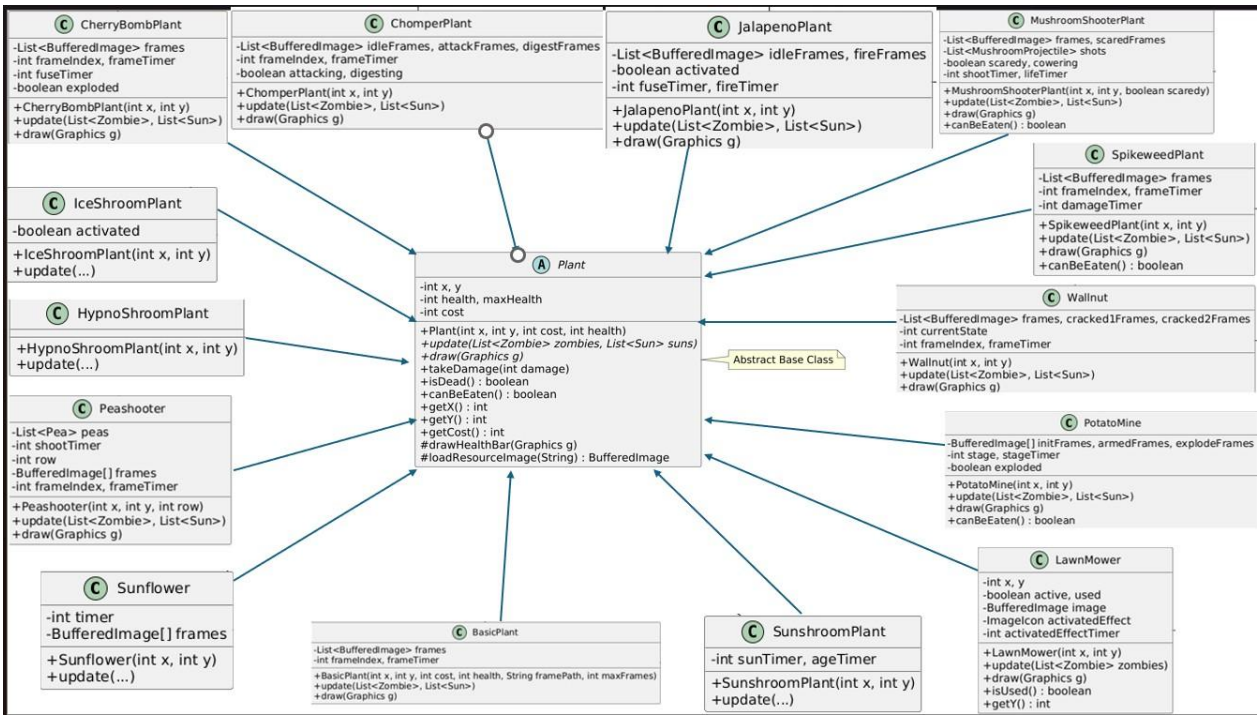
- Plant (Abstract Class): Peashooter, Sunflower, Wall nut, Potato Mine, Cherry Bomb Plant, Chomper Plant, Jalapeno Plant, Ice Shroom Plant, Hypno Shroom Plant, Mushroom Shooter Plant, Sunshroom Plant, Spikeweed Plant, Basic Plant

- Zombie (Base Class)

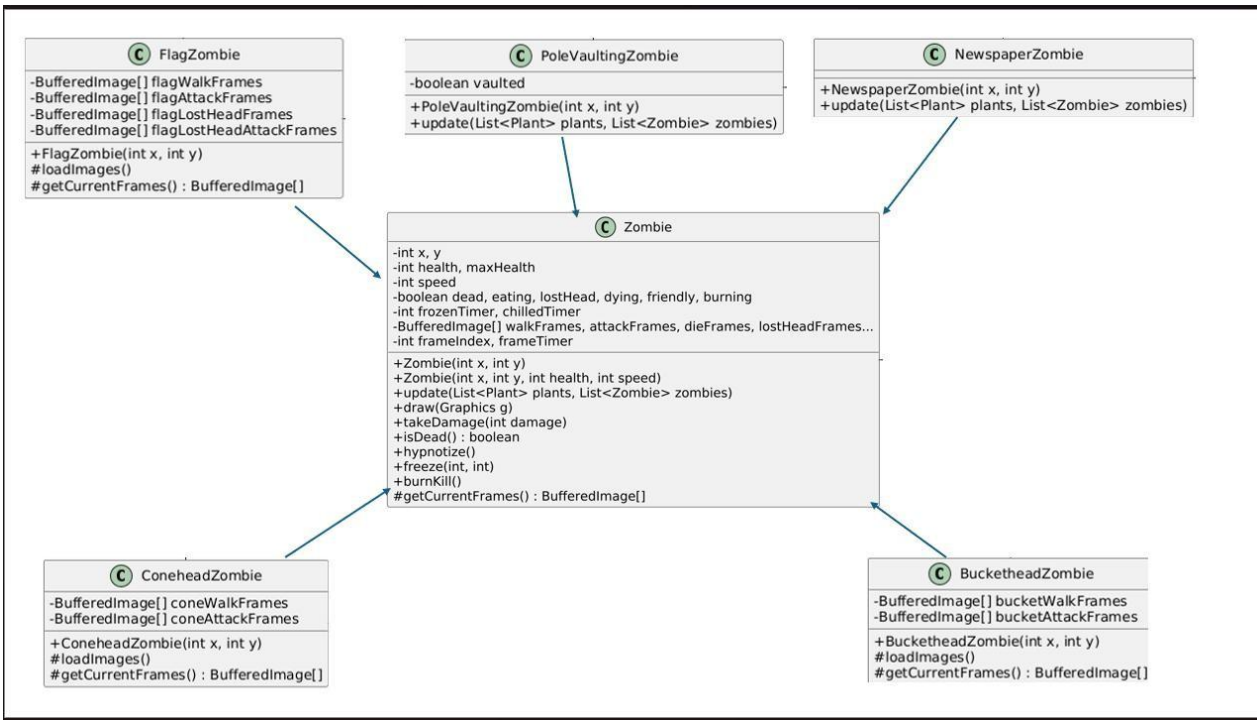
Normal Zombie, Conehead Zombie, Buckethead Zombie, Flag Zombie, Pole Vaulting Zombie, Newspaper Zombie

Supporting Classes: Pea, Mushroom Projectile, Sun, Lawn Mower, Game Panel, Menu Panel, Zombie Stats

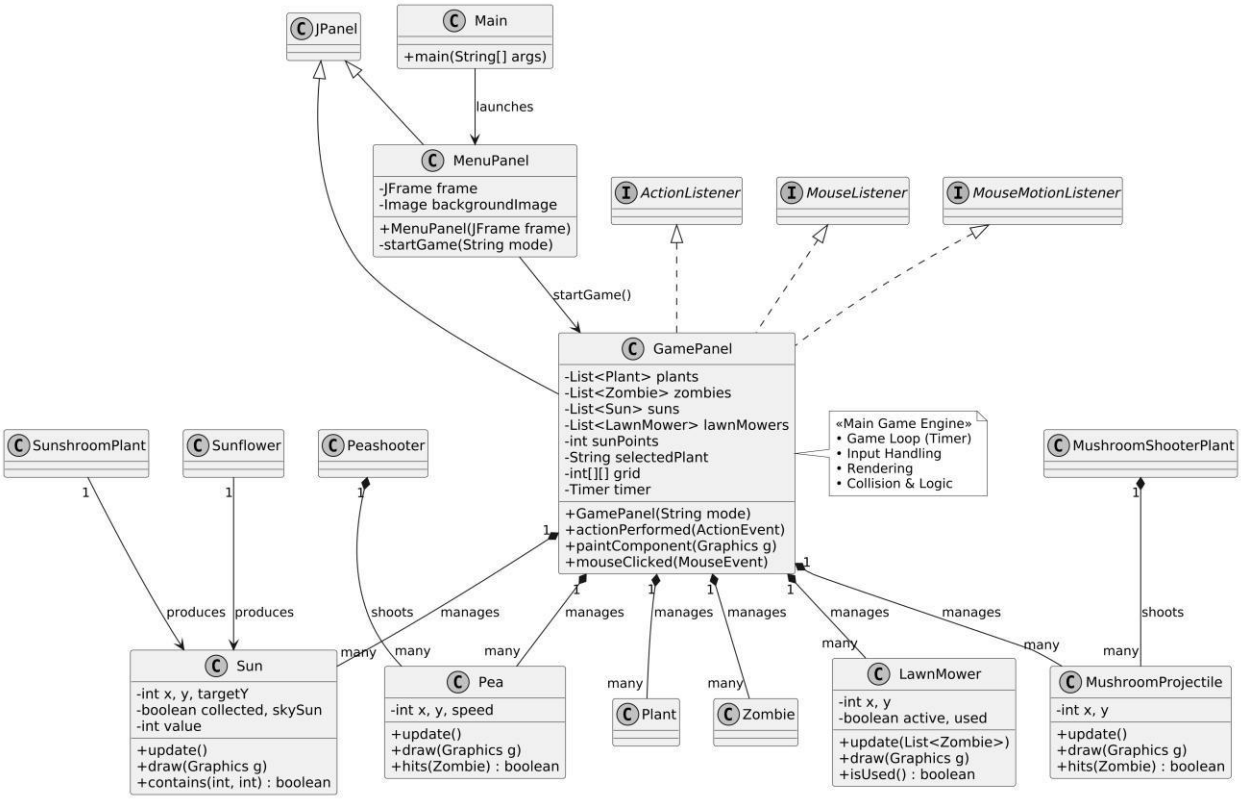
(Class Diagram - Plants)



(Class Diagram - Zombies)



(Class Diagram - Game Core & Supporting Classes)



3.2 Design Patterns Used

- Inheritance & Polymorphism: Heavy use in **Plant** and **Zombie** hierarchies.

- Composition: **GamePanel** contains lists of plants, zombies, suns, etc.

- Factory-like pattern (partial): Different zombie types are created with different properties.

4. Implementation

4.1 Technologies Used -

Language: Java

- GUI Framework: Swing

- Image Handling: **BufferedImage** + **ImageIcon**

- Animation: Frame-based animation with timers

- Event Handling: MouseListener, ActionListener, Timer

4.2 Key Features Implemented

- Main Menu with Day/Night mode selection
- Fully functional grid-based planting system with hover feedback
- 13+ different Plant types with unique behaviors
- 6 different Zombie types with special abilities- Advanced shooting mechanics (Pea + Mushroom Projectile)
- Sun spawning, collection, and management system
- Plant cooldown and seed packet system
- Shovel tool to remove plants
- Multiple special abilities (Explosion, Freeze, Hypnotize, Fire line, Eating, etc.)
- Lawn Mower defense system
- Smooth animation for all entities
- Day and Night background modes with different plant rosters
- Collision detection and health/damage system

4.3 Important Classes

- **GamePanel.java** — Main game engine (game loop, rendering, input handling)
- **Plant.java** — Abstract base class for all plants
- **Zombie.java** — Base class with animation states and special effects
- **Peashooter.java** — Shooting logic
- **Sunflower.java / SunshroomPlant.java —** Resource generation
- **LawnMower.java** — Emergency lane defense
- **MenuPanel.java** — Enhanced menu with hover effects

5. Challenges & Solutions

1. Animation Performance

- Challenge: Smooth animation with many entities.
- Solution: Used frame counters and **Graphics2D** with bicubic interpolation.

2. Collision Detection

- Challenge: Accurate hit detection between many object types.
- Solution: Combined distance-based and bounding box checks.

3. Resource Loading

- Challenge: Loading hundreds of image assets reliably.
- Solution: Robust **loadResourceImage()** method with multiple path fallbacks and background removal.

4. Complex Plant Behaviors

- Challenge: Managing different states (idle, attacking, exploding, digesting).
- Solution: Used internal state variables and polymorphism.

5. Game Loop

- Challenge: Managing updates and rendering consistently. -
Solution: Central **Timer** (30ms) in **GamePanel**.

6. Future Improvements (Bonus Features)

- Sound effects and background music
- More levels with increasing difficulty and boss waves
- Save/Load game progress
- Additional plant and zombie types
- Win condition (survive a certain number of waves)
- Particle effects and better visual feedback

7. Conclusion

This project successfully demonstrates the implementation of a complex 2D tower defense game using Java Swing. Through effective use of OOP principles such as inheritance, polymorphism, and encapsulation, we created a functional, visually appealing, and highly playable version of Plants vs. Zombies.

The development process significantly improved our understanding of game programming, GUI development, event handling, and software design. The codebase is modular and extensible, allowing for easy addition of new features in the future.